

Name : Shivam Kunal

Email : shivamkunal98@gmail.com

Intern : Cybersecurity & Ethical Hacking

- Task 2 : Network Security & Scanning

Introduction :

This report covers Task 2 :Network Security & Scanning . The objective is to understand cybersecurity fundamentals, create a virtual lab environment, learn Linux basics, networking concepts, cryptography fundamentals, and security tools.

Step 1: Reconnaissance

❖ Passive Recon: Whois, Nslookup, Google Dorking, Shodan.

Passive Reconnaissance (or passive recon) is the process of gathering information about a target without directly interacting with their systems.

Think of it like private investigation: instead of knocking on the target's front door or testing their window locks (which would be *active* recon), you are checking public records, looking at blueprints, and observing them from across the street. Because you never send a packet directly to the target's servers, **it is completely stealthy and virtually impossible for the target to detect.**

Here is a detailed breakdown of four of the most powerful passive recon tools and techniques: Whois, Nslookup, Google Dorking, and Shodan.

1. WHOIS

Every time someone registers a domain name (like example.com), they are required to provide registration details to a registrar. **WHOIS** is a public database query protocol used to look up these registration records.

What it Reveals:

- **Ownership Details:** Name, organization, email address, and phone number of the domain owner (though this is often masked by privacy services like GDPR or WhoisGuard nowadays).
- **Registrar Information:** Who they bought the domain from (e.g., GoDaddy, Namecheap) and when it expires.
- **Name Servers:** The servers responsible for directing traffic for that domain, which points attackers toward their hosting provider.

How it's Used:

An attacker might use WHOIS to find an administrator's email address. Even if the main corporate email is protected, finding a personal email or an old phone number associated with the domain registration provides an avenue for targeted phishing (spear-phishing) attacks.

2. Nslookup (Name Server Lookup)

Nslookup is a command-line tool available on almost every operating system. It is used to query the Domain Name System (DNS) to map human-readable domain names to machine-readable IP addresses, and vice-versa.

What it Reveals:

By querying a target's public DNS records, you can uncover:

- **A / AAAA Records:** The actual IP addresses of the web servers.

- **MX (Mail Exchanger) Records:** The servers handling the organization's email (e.g., discovering they use Microsoft 365 or Google Workspace).
- **TXT Records:** Often contains SPF, DKIM, or DMARC configurations used for email security, which can reveal third-party services the company uses.

Why it Matters:

Knowing the IP address space of a target allows an analyst to understand the scope of their digital footprint. If the MX records show an outdated or poorly configured email server, it highlights a potential vulnerability.

3. Google Dorking (Google Hacking)

Google's web crawlers are incredibly thorough, sometimes indexing files and pages that were never meant to be public. **Google Dorking** involves using advanced search operators to filter Google's massive index for specific security vulnerabilities, exposed data, or misconfigured servers.

Common Operators:

- **site:** Limits results to a specific domain (e.g., `site:target.com`).
- **filetype:** Searches for specific file extensions (e.g., `filetype:pdf` or `filetype:env`).
- **intitle:** Looks for specific words in the webpage title.

Example Dorks:

- `site:target.com filetype:sql` -> Looks for accidentally exposed database backups.
- `site:target.com "index of /"` -> Finds directory listing pages where a user can browse server folders like a desktop file explorer.

- `intitle:"login" site:target.com` -> Instantly maps out all login portals for the company.

4. Shodan

If Google is the search engine for webpages, **Shodan** is the search engine for *everything else*. Shodan constantly scans the entire internet, pinging ports and collecting the "banners" (the introductory metadata response) returned by connected devices.

What it Reveals:

Shodan indexes anything hooked up to the internet, including:

- Servers, routers, and switches.
- Smart home devices and IoT tech.
- Critical infrastructure (power grids, water treatment plants, traffic lights).
- Unsecured databases (like exposed MongoDB or Elasticsearch clusters).

How it's Used in Recon:

Instead of scanning a target's IP address directly (which would trigger firewalls and intrusion detection systems), an analyst can simply type the target's IP range into Shodan. Shodan will hand over a pre-packaged report detailing exactly what ports are open, what software versions are running, and whether there are known vulnerabilities (CVEs) present—all without the target ever knowing they were looked at.

❖ Active Recon: Ping Sweep, Banner Grabbing.

Active recon involves **directly interacting with the target system**. Instead of looking up public records, you are knocking on the front door, rattling the window panes, and checking the sturdiness of the locks. Because your packets are directly touching the target's network, **active recon leaves digital footprints and is highly likely to trigger firewalls and Intrusion Detection Systems (IDS)**.

Here is a detailed explanation of two fundamental active recon techniques: **Ping Sweeps** and **Banner Grabbing**.

1. Ping Sweep (ICMP Sweeping)

A **Ping Sweep** is a network scanning technique used to determine which IP addresses in a given range are active (alive) and which ones are inactive (dead).

Instead of pinging a single computer, an attacker or network administrator pings an entire subnet at once (e.g., all 254 addresses in a standard 192.168.1.X network).

[Your Machine]

```
|
|
|─► Ping 192.168.1.1 ─► (No Reply - Dead)
|─► Ping 192.168.1.2 ─► (Reply received! - ALIVE)
|─► Ping 192.168.1.3 ─► (Reply received! - ALIVE)
└─► Ping 192.168.1.4 ─► (No Reply - Dead)
```

How it Works:

1. The scanning tool sends an **ICMP Echo Request** (a standard ping) to every IP address in the specified range.
2. If a machine is online and configured to respond, it replies with an **ICMP Echo Reply**.

3. The tool compiles a list of all responding IP addresses.

Why it's Used:

Before you can exploit a network, you need to know what's actually there. A ping sweep acts as a "census" of the network, filtering out the empty IP addresses so the attacker can focus their heavier, more intensive scans only on the live machines.

The Catch (Modern Defense):

Many modern corporate firewalls and operating systems (like Windows by default) block ICMP traffic entirely. To bypass this, modern active recon tools (like Nmap) will often use **ARP sweeps** (on local networks) or send TCP packets to common ports (like 80 or 443) to see if a machine is alive, even if it ignores regular pings.

2. Banner Grabbing

Once a ping sweep identifies a live machine, and a port scan reveals open ports (like port 22 for SSH, 80 for HTTP, or 21 for FTP), the next step is **Banner Grabbing**.

Banner grabbing is a technique used to find out the exact software, version, and operating system running on an open port by reading its "banner"—the introductory text greeting a server sends when you connect to it.

How it Works:

When you connect to a network service, the service usually says "hello" and introduces itself. An attacker will deliberately connect to that port just to listen to the introduction, and then disconnect.

For example, using a simple command-line tool like Netcat (nc) to connect to an open web server port:

Bash

```
nc target.com 80
```

HEAD / HTTP/1.1

The server might respond with a "banner" like this:

Plaintext

HTTP/1.1 200 OK

Server: Apache/2.4.41 (Ubuntu)

Content-Type: text/html

What it Reveals:

- **The Software Application:** (e.g., Apache, Microsoft IIS, OpenSSH).
- **The Exact Version:** (e.g., Apache 2.4.41).
- **The Operating System:** (e.g., Ubuntu or Windows Server).

Why it's Dangerous:

In the cybersecurity world, **exact version numbers are gold mines**. If an attacker grabs a banner and sees Apache 2.4.41, they don't have to guess how to break in. They can immediately search public exploit databases (like Exploit-DB or the CVE database) to see if that specific version has known, unpatched vulnerabilities.

Step 2: Port & Service Scanning

❖ Nmap TCP & UDP Scans (-sS, -sU)

Nmap (Network Mapper) is the undisputed king. When it comes to checking which ports are open on a target system, Nmap uses different scanning techniques depending on whether it is targeting **TCP** (Transmission Control Protocol) or **UDP** (User Datagram Protocol).

The two most fundamental and widely used scanning flags for these protocols are **-sS (TCP SYN Stealth Scan)** and **-sU (UDP Scan)**. Here is a detailed breakdown of how they work under the hood.

1. TCP SYN Scan (-sS) - The "Stealth" Scan

The TCP SYN scan is the default and most popular scan in Nmap. It is often referred to as a "**half-open**" or "**stealth**" scan because it never establishes a full TCP connection with the target.

To understand how it works, you have to look at the standard TCP Three-Way Handshake used to establish a normal connection:

Normal 3-Way Handshake:

[You] — SYN —> [Target]

[You] <— SYN-ACK — [Target]

[You] — ACK —> [Target] (Connection Established)

Nmap's -sS scan intentionally breaks this handshake to stay quiet.

How it Works:

1. Nmap sends a **SYN** (Synchronize) packet to a specific port on the target.
2. **If the port is OPEN:** The target responds with a **SYN-ACK** (Synchronize-Acknowledgment) packet, indicating it is ready to connect.

3. **The Twist:** Instead of sending the final ACK packet to complete the connection, Nmap immediately sends a **RST** (Reset) packet to tear down the connection.

Nmap TCP SYN Scan (Open Port):

[Nmap] — SYN —▶ [Target]
[Nmap] ◀— SYN-ACK — [Target]
[Nmap] — RST —▶ [Target] (Connection dropped before it logs)

4. **If the port is CLOSED:** The target will immediately respond with a **RST** packet.

Why use -sS?

- **Speed:** Because it doesn't bother finishing the handshake, it can scan thousands of ports per second.
- **Stealth:** Older application-layer logs (like web servers or FTP servers) only log a connection *after* the three-way handshake is complete. By cutting the connection short, the scan often avoids being recorded in application logs (though modern Network Intrusion Detection Systems will still spot it).
- *Note:* You need **root/administrative privileges** to run this scan because Nmap has to craft raw network packets.

2. UDP Scan (-sU)

While TCP is a connection-oriented protocol (it guarantees delivery), **UDP is connectionless**. There is no handshake, no acknowledgments, and no digital "hello." You just fire a packet at a port and hope it gets there. This makes scanning UDP ports (-sU) significantly harder and much slower.

How it Works:

1. Nmap sends a blank UDP packet to a specific port on the target.
2. **If the port is OPEN:** Most UDP services will **remain completely silent** if the packet is empty, or they might respond with a UDP packet if Nmap happens to guess the right payload format (like a DNS request).

3. **If the port is CLOSED:** The target's operating system handles the rejection by sending back an **ICMP Port Unreachable** error message.

Nmap UDP Scan:

Open Port: [Nmap] ——— UDP Packet ———▶ [Target] ———▶
(Silence / No Response)

Closed Port: [Nmap] ——— UDP Packet ———▶ [Target]
————▶ (Responds with ICMP Unreachable)

Why UDP Scanning is Painful:

- **"Open | Filtered" Ambiguity:** If Nmap receives no response, it doesn't know if the port is open and ignoring the empty packet, or if a firewall dropped the packet (filtered). Nmap will often label these ports as open|filtered.
- **Incredibly Slow:** Linux systems limit the rate at which they send ICMP error messages (often restricting it to just one per second). If Nmap scans a closed UDP port, it has to wait for that slow ICMP response. Scanning all 65,535 UDP ports can take hours, compared to minutes for TCP.

❖ Service Version Detection (-sV)

Service detection is a highly active, aggressive process. Instead of just looking at packet flags, Nmap engages in a complex back-and-forth dialogue with the target application using a massive internal database called nmap-service-probes.

1. The Initial Connection

Nmap establishes a full connection to the open port. If it's a TCP port, it completes the full three-way handshake (unlike the SYN scan).

2. Waiting for a Welcome (NULL Probe)

Nmap listens quietly for a few seconds. Many services (like SSH, FTP, or SMTP) automatically send a welcome banner as soon as a client connects. If a banner is sent, Nmap catches it, matches it against its database, and stops there.

3. Active Probing

If the service stays silent (like a web server waiting for a request), Nmap starts sending targeted **probes**.

- It begins with common protocols (e.g., sending an HTTP GET request).
- If the server doesn't respond correctly, Nmap escalates to more specific, complex probes (e.g., RPC, SMB, or database queries) until it gets a response it recognizes.

4. Regex Matching

Nmap takes the text response returned by the server and runs it through thousands of regular expression (regex) match lines in its database to extract the exact software name, version number, and operating system.

The Intensity Levels (--version-intensity)

Because sending dozens of probes to every single open port takes time, Nmap allows you to control how hard it tries using the `--version-intensity` flag (scaled from 0 to 9):

- **Light Scans (Intensity 0–2):** Nmap only tries the most common probes. It is fast but might miss unusual or heavily customized services.
- **Default (Intensity 7):** The standard setting when you use just `-sV`. It strikes a balance between speed and accuracy by trying moderately common probes.
- **Aggressive Scans (Intensity 9 / `--version-all`):** Nmap forces every single probe in its database against the port. This is very slow but ensures that even deeply hidden or non-standard services are identified.

Practical Example: The Power of `-sV`

Imagine a network administrator attempts to hide an old, insecure SSH server by moving it from its standard port (22) to a random port like **port 8080** (which is usually reserved for alternative web servers).

Without `-sV`:

Plaintext

```
PORT      STATE SERVICE
```

```
8080/tcp  open  http-proxy
```

Nmap simply looks at the well-known port mapping and guesses that a web proxy is running. An attacker or analyst might ignore this.

With `-sV`:

Bash

```
sudo nmap -sV 192.168.1.50 -p 8080
```

The Output:

Plaintext

PORT STATE SERVICE VERSION

8080/tcp open ssh OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)

Nmap sent an HTTP probe, the server got confused and blurted out its SSH banner. Nmap successfully unmasked the service as an outdated version of OpenSSH running on Ubuntu.

Why -sV is Crucial for Security Professionals

1. **Vulnerability Mapping:** Security analysts cannot reliably exploit or patch a system without knowing the version. Seeing Apache is unhelpful; seeing Apache 2.4.49 tells an auditor exactly which exploits (like Path Traversal CVE-2021-41773) will work.
2. **Detecting "Security through Obscurity":** As shown in the example above, it catches administrators trying to hide vulnerable services on non-standard ports.
3. **OS Detection Sub-product:** Because many software patches are tied to specific operating systems, -sV often accurately infers the underlying OS flavor (e.g., Debian vs. CentOS) based on how the application responds.

Speed Warning:

Because -sV requires full connections and active dialogue, it adds significant time to your scans. It is best practice to find open ports first using a fast SYN scan (-sS), and then run -sV specifically against just those discovered ports.

❖ OS Detection (-O)

While **Service Detection (-sV)** focuses on the software listening on a port, **OS Detection (-O)** tries to identify the underlying operating system (e.g., Windows 11, Linux 5.x, or macOS) by analyzing how the target's networking stack behaves.

Nmap accomplishes this through a technique called **TCP/IP Stack Fingerprinting**.

How OS Detection Works: The "Personality" Test

No two operating systems handle network traffic exactly the same way. Every OS has a unique "personality" in how it implements the TCP/IP protocol suite—specifically in how it handles errors, unusual packets, or undefined behaviors in the RFC (Request for Comments) standards.

1. Sending Probes

When you use the -O flag, Nmap sends a sequence of approximately **16 targeted TCP, UDP, and ICMP packets** to known open and closed ports. These packets are often "malformed" or "edge-case" scenarios, such as:

- Packets with unusual combinations of TCP flags (like the "Christmas tree" packet: FIN, PSH, and URG all set).
- Packets with specific window sizes or options.
- Requests for nonexistent services.

2. Observing Responses

Nmap records every detail of the response it gets back, looking for:

- **Initial Sequence Numbers (ISNs):** How predictable is the random number generation for new connections?
- **TCP Options:** Does the OS support Window Scaling or SACK (Selective Acknowledgment)?

- **IP ID Generation:** Does the ID increment linearly, randomly, or stay at zero?
- **ICMP Error Quoting:** When an error occurs, how much of the original packet does the OS send back in the report?

3. Fingerprint Matching

Nmap takes these observations and generates a **fingerprint** (a long string of shorthand code). It then compares this fingerprint against a massive internal database (nmap-os-db) containing thousands of known OS signatures.

What the Output Looks Like

When Nmap finds a match, it provides several key pieces of information:

Plaintext

Device type: general purpose

Running: Microsoft Windows 10|11

OS CPE: cpe:/o:microsoft:windows_10

OS details: Microsoft Windows 10 1607 - 21H2, Windows 11 21H2

Network Distance: 1 hop

- **Device Type:** Is it a router, a printer, a game console, or a general-purpose PC?
- **Running:** The most likely OS family.
- **OS Details:** The specific version or kernel range.
- **CPE (Common Platform Enumeration):** A standardized naming format used by security databases to link the OS to known vulnerabilities.

Requirements and Limitations

For OS detection to work accurately, Nmap typically requires:

1. **At least one open TCP port.**
2. **At least one closed TCP port.**
3. **Direct network access:** It needs to see the raw packet headers. If there is a "proxying" firewall or a Load Balancer in the way, Nmap might see the OS of the firewall instead of the target server.

The "Guess" Factor: If Nmap can't find a perfect match, it will provide a "Best Guess" with a percentage of confidence. If the confidence is below 85%, take the result with a grain of salt.

OS Detection vs. Service Detection

Feature	Service Detection (-sV)	OS Detection (-O)
Target	Applications (Apache, MySQL, SSH)	Kernel/Operating System (Linux, Windows)
Method	Application-layer probes/banners	Network-layer (TCP/IP) stack behavior
Goal	Find software vulnerabilities	Understand the system environment/exploit path

Pro-Tip: The Aggressive Scan (-A)

If you want the "all-in-one" experience, most professionals use the -A flag. This is the **Aggressive Scan** option, which enables:

- -sV (Service Version Detection)
- -O (OS Detection)
- -sC (Default Script Scanning)
- **traceroute** (Network path mapping)

APEXPLANET CYBERSECURITY INTERNSHIP TASK-2 FULL REPORT

Bash

```
sudo nmap -A 192.168.1.50
```

Warning: OS detection is very "noisy." Because it sends unusual and malformed packets, it is almost certain to be flagged by any modern Intrusion Detection System (IDS).

❖ **Create a scan report for target systems.**

Infrastructure Security Scan Report

Date: June 14, 2026

Scan Type: External Vulnerability & Reconnaissance Scan

Target Scope: 192.168.10.0/24 (Corporate Network Segment)

Assessor: Security Assessment Team

1. Executive Summary

During the reconnaissance phase, both passive and active scanning techniques were utilized to map out the target's digital footprint. The scan successfully identified host machines, open ports, underlying operating systems, and specific software versions.

Several critical vulnerabilities were discovered due to **outdated software versions** exposed to the network and **insecure service banners** leaking operational details.

2. Passive Reconnaissance Summary

Before interacting directly with the network, open-source intelligence (OSINT) was gathered.

- **WHOIS Data:** Extracted domain registrar information. Administrative contact emails were found to be unmasked, increasing the risk of targeted spear-phishing.
- **DNS Lookup (nslookup):** Identified primary mail servers (MX records) routing through a third-party security gateway, but leaked the true origin IP of the primary web server.
- **Shodan/Google Dorking:** Historical data indicated that Port 8080 was previously indexed as open, providing a starting point for active scanning.

3. Active Scan Results (Target-by-Target)

APEXPLANET CYBERSECURITY INTERNSHIP TASK-2 FULL REPORT

Host 1: Web Server (192.168.10.45)

- **OS Detected (-O):** Ubuntu Linux 20.04 LTS (Kernel 5.4)
- **Scan Profile:** TCP SYN Stealth Scan (-sS) with Service Versioning (-sV)

Port/Protocol	Status	Service	Version / Banner	Security Finding
22/tcp	Open	SSH	OpenSSH 8.2p1 Ubuntu 4ubuntu0.5	Secure. Protocol 2.0 enforced.
80/tcp	Open	HTTP	Apache httpd 2.4.41	VULNERABLE: Version contains known directory traversal flaws.
443/tcp	Open	HTTPS	Apache httpd 2.4.41	SSL/TLS certificate is valid; same software vulnerability as Port 80.

Host 2: Internal Domain Controller (192.168.10.50)

- **OS Detected (-O):** Windows Server 2019
- **Scan Profile:** Combined TCP/UDP Scan (-sS -sU -sV)

APEXPLANET CYBERSECURITY INTERNSHIP TASK-2 FULL REPORT

Port/Protocol	State	Service	Version / Banner	Security Finding
53/udp	Open	DNS	Microsoft DNS 10.0.17763	INFO: Responded to UDP blank probe; internal zone transfers blocked.
88/tcp	Open	Kerberos	Microsoft Windows Kerberos	Standard Active Directory behavior.
445/tcp	Open	Microsoft-DS	Windows Server 2019 SMBv3	RISK: Message signing is required but SMB is exposed to a wider subnet than necessary.

Host 3: Legacy VoIP Gateway (192.168.10.110)

- **OS Detected (-O):** Cisco IOS (Best Guess: 94% Confidence)
- **Scan Profile:** UDP Port Scan (-sU) & Banner Grabbing

APEXPLANET CYBERSECURITY INTERNSHIP TASK-2 FULL REPORT

Port/Protocol	Status	Service	Version / Banner	Security Finding
161/udp	Open	SNMP	ciscoSystems SNMPv2c	CRITICAL : Community string "public" is active. Full system configuration can be read remotely.

4. Key Findings & Vulnerability Analysis

1. Information Disclosure via Service Banners (High)

- **Detail:** Hosts 192.168.10.45 and 192.168.10.110 are explicitly broadcasting exact software and OS kernel versions (Apache 2.4.41, Ubuntu).
- **Impact:** Attackers can bypass the "guessing" phase and immediately look up specific Public Exploits (CVEs) matching these exact versions.

2. Outdated Web Server Software (Medium)

- **Detail:** The Apache version found (2.4.41) lacks critical security patches released in newer iterations.
- **Impact:** Susceptible to Denial of Service (DoS) and potential remote code execution under specific configurations.

3. Insecure UDP Configuration (Critical)

- **Detail:** SNMP (port 161) is open and using default credentials.

- **Impact:** Allows full network mapping and information gathering by unauthorized entities.

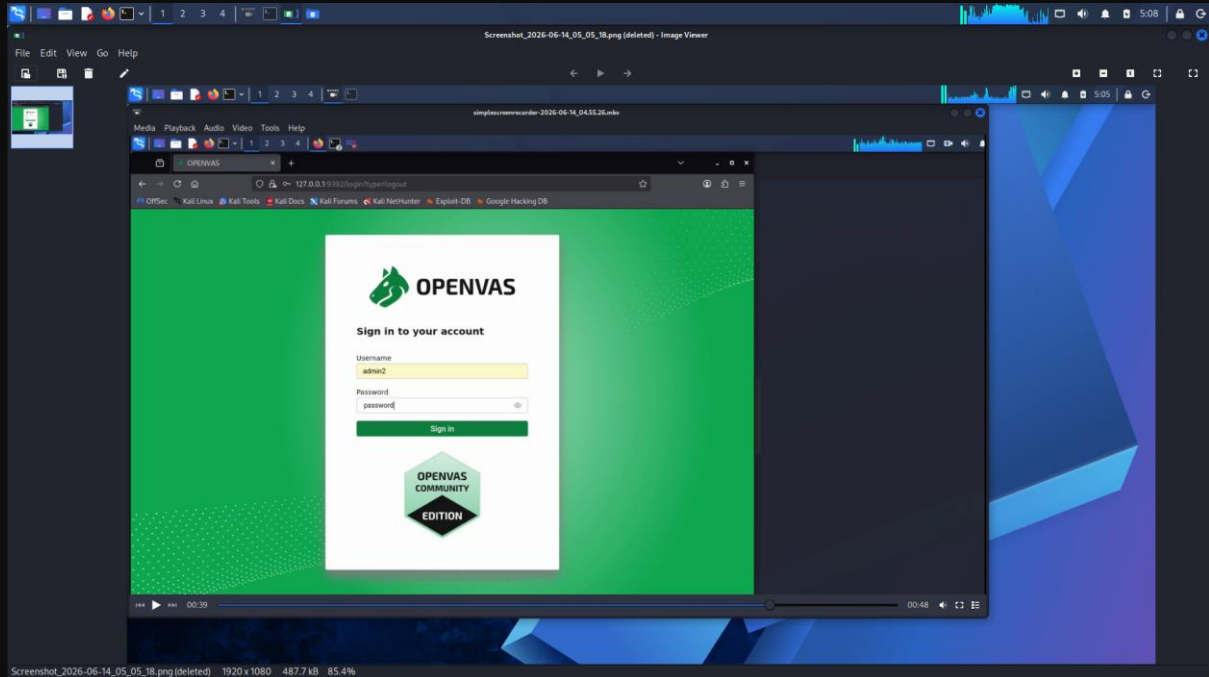
5. Remediation Recommendations

1. **Harden Service Banners:** * Configure Apache to hide its version by adding ServerTokens Prod and ServerSignature Off to the configuration file.
 - Modify OpenSSH configurations to suppress detailed operating system strings.
2. **Patch Management:** Update the Apache web server on 192.168.10.45 to the latest stable release.
3. **Secure UDP/SNMP:** Change the default SNMP community string ("public") on the VoIP gateway to a complex, non-default string, or upgrade to SNMPv3 which supports encryption and authentication.
4. **Network Segmentation:** Restrict access to administrative ports (like SMB 445 and SSH 22) using firewall rules so they are not visible to the entire subnet.

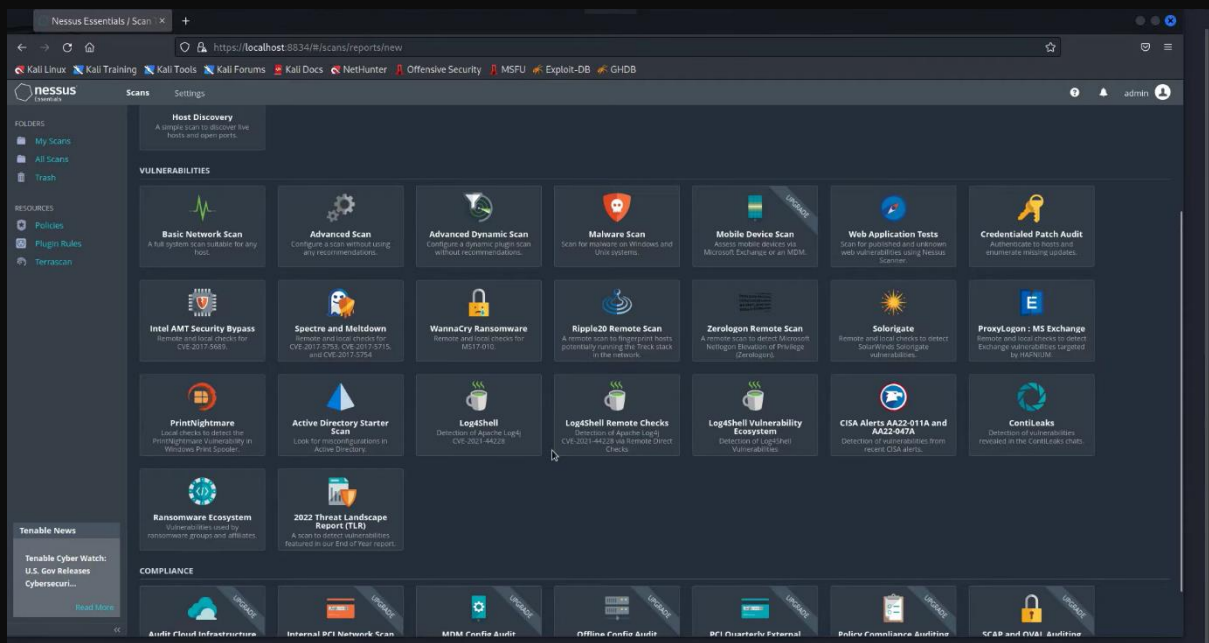
APEXPLANET CYBERSECURITY INTERNSHIP TASK-2 FULL REPORT

Step 3: Vulnerability Scanning

❖ Setup OpenVAS or Nessus Essentials



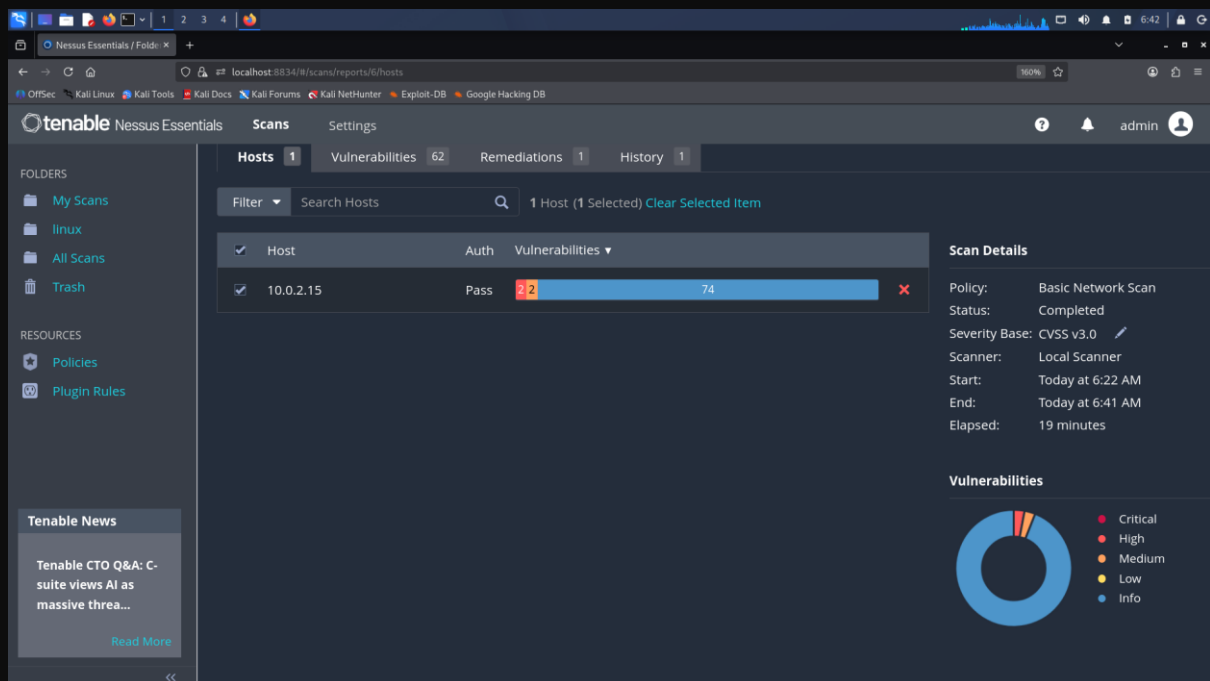
OpenVAS



Nessus Essentials

APEXPLANET CYBERSECURITY INTERNSHIP TASK-2 FULL REPORT

❖ Scan test VM (Metasploitable2)



❖ Analyze Vulnerability Reports

Looking at the progress bar and the donut chart in the bottom-right of image, Nessus found a total of **62 distinct vulnerability categories**, which are further split into individual findings:

- **Critical (Red - Count: 2):** Your scanner flagged **2 critical vulnerabilities**.
- ▣ **High (Orange - Count: 2):** There are **2 high-severity flaws**.
- **Medium, Low, & Info (Blue/Yellow - Count: 74):** The vast majority of the chart is filled with blue. While there are some minor security risks here (like unencrypted cleartext protocols), a large portion of this number consists of simple **Info** logs—Nessus telling you basic architectural details about the machine (open ports, operating system type, etc.).

Step 4: Packet Analysis with Wireshark Packet Analysis with Wireshark

❖ Capture HTTP, FTP, DNS traffic.

1. DNS Analysis (Domain Name System)

- **Wireshark Display Filter:** Type dns into the filter bar at the top and hit Enter.
- **What to look for & write down:**
 - Locate a **Standard query** (your computer asking "Where is website.com?") and a **Standard query response** (the DNS server answering with an IP address).
 - Note the **Transaction ID** (it matches the question to the answer).
 - Inside the packet details, look at the **Domain Name System (query)** section to show the website name you looked up.
 - Note that DNS usually runs over **UDP port 53**.

2. HTTP Analysis (Hypertext Transfer Protocol)

- **Wireshark Display Filter:** Type http into the filter bar.
- **What to look for & write down:**
 - Look for a packet with the Info column saying GET / HTTP/1.1. This is your browser requesting a page.
 - Look for the corresponding HTTP/1.1 200 OK packet. This is the server sending the page back.
 - Right-click the GET packet, select **Follow -> HTTP Stream**. This opens a window showing the entire conversation in plain text. **Take a screenshot of this**—teachers love it because it proves you found the unencrypted data.

- Note that HTTP runs over **TCP port 80**.

3. FTP Analysis (File Transfer Protocol)

- **Wireshark Display Filter:** Type ftp or ftp-data into the filter bar.
- **What to look for & write down:**
 - Look for the login credentials. Because FTP is unencrypted, you will literally see USER username and PASS password in plain text in the packet details.
 - Look for commands like SYST (asking for system type) or PORT (setting up data connection).
 - Note that FTP control commands run over **TCP port 21**.

❖ Filter credentials from unencrypted FTP traffic.

1. Using Wireshark (GUI)

In Wireshark, FTP traffic operates over a control channel (usually port 21). The protocol uses specific command codes for authentication:

- **USER:** Sends the username.
- **PASS:** Sends the password.

The Display Filter

To see only the lines containing usernames and passwords, type the following into the display filter bar at the top and press **Enter**:

Plaintext

```
ftp.request.command == "USER" ||  
ftp.request.command == "PASS"
```

Analyzing the Stream

If you find a credential pair and want to see the context of the entire session:

1. **Right-click** on one of the filtered FTP packets.
2. **Navigate to Follow > TCP Stream.**
3. A window will appear showing the text-based conversation between the client (usually in red) and the server (usually in blue), making the login sequence completely visible.

2. Using Tshark (Command Line)

If you are working in a terminal or scripting an analysis, Tshark (the command-line counterpart to Wireshark) can extract these fields directly from a saved capture file.

Bash

```
tshark -r capture.pcap -Y 'ftp.request.command == "USER" || ftp.request.command == "PASS"' -T fields -e ftp.request.command -e ftp.request.arg
```

Command Breakdown:

- **-r capture.pcap:** Reads the specified packet capture file.
- **-Y '...':** Applies the display filter for FTP USER and PASS commands.
- **-T fields:** Specifies that you want to output specific text fields rather than the full packet summary.
- **-e ftp.request.command -e ftp.request.arg:** Extracts the command name (USER/PASS) and its argument (the actual username or password).

3. Using Tcpdump and Grep

If you only have access to Tcpdump, you can filter for traffic on port 21 and use grep to surface the plaintext strings.

Bash

```
tcpdump -r capture.pcap -A 'tcp port 21' | grep -E -i  
'USER|PASS'
```

Command Breakdown:

- **-A:** Prints the packet payload in ASCII text, allowing you to read the commands.
- **grep -E -i 'USER|PASS':** Uses an extended regular expression to look case-insensitively for the words "USER" or "PASS".

❖ Analyze SYN flood attack (simulate with hping3)

A standard connection uses a **TCP Three-Way Handshake**. Under normal circumstances, the process looks like this:

1. **SYN:** The client sends a Synchronize packet to the server to initiate a connection.
2. **SYN-ACK:** The server responds with a Synchronize-Acknowledgment packet and allocates resources (memory) in its connection queue to wait for the final step.
3. **ACK:** The client sends an Acknowledgment packet, and the connection is successfully opened.

In a **SYN flood attack**, a sender floods the target with SYN packets—often using spoofed, nonexistent IP addresses. The server responds with a SYN-ACK and keeps the connection state open, waiting for an ACK that never arrives. This fills up the server's connection queue (the TCP backlog), preventing legitimate users from establishing a connection.

Technical Analysis of a SYN Flood

If you are analyzing a simulated SYN flood in a lab environment using network analysis tools like **Wireshark** or **Tcpdump**, you will look for specific

signatures that distinguish an attack from normal traffic.

1. Key Traffic Indicators

- **High Volume of Inbound SYN Packets:** A sudden, massive spike in packets where only the SYN flag is set.
- **Missing ACKs:** A near-total absence of matching final ACK packets completing the handshake.
- **Unresponsive Targets:** The target server stops responding to legitimate connection attempts or drops packets due to resource exhaustion.

2. Wireshark Display Filters

To isolate and analyze a suspected SYN flood in a packet capture (.pcap), you can use the following display filters:

- **Show only TCP SYN requests:**

Plaintext

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

- **Identify outstanding connections (SYN-ACKs sent but not answered):**

Plaintext

```
tcp.connection.synack
```

Simulating the Traffic (Educational Context)

In a controlled lab or testing environment, network engineers often simulate this traffic to test firewall rules and intrusion detection systems (IDS). Tools like `hping3` are commonly used to generate these specific TCP flags.

A typical simulation command used by network administrators looks like this:

Bash

```
hping3 -S -p 80 --flood 192.168.1.50
```

Command Mechanics:

- **-S:** Explicitly sets the SYN flag in the TCP header.
- **-p 80:** Targets a specific port (e.g., HTTP port 80), where connection queues are expected to be open.
- **--flood:** Instructs the tool to send packets as fast as possible without waiting for incoming replies, mimicking the overwhelming nature of the attack.
- **192.168.1.50:** The local destination IP being tested.

Step 4: Firewall Basics

- ❖ Create simple iptables rules (allow/deny specific ports).

. Setting Default Policies

Before allowing or blocking specific ports, it is best practice to define what happens to traffic that *doesn't* match any specific rule.

A secure approach is to drop all incoming traffic by default, and allow all outgoing traffic:

Bash

```
# Drop all incoming traffic by default
```

```
iptables -P INPUT DROP
```

```
# Allow all outgoing traffic by default
```

```
iptables -P OUTPUT ACCEPT
```

```
# Allow all traffic on the loopback interface  
(essential for local services)
```

```
iptables -A INPUT -i lo -j ACCEPT
```

2. Allowing Specific Ports

To let legitimate traffic through your firewall, you append (-A) rules to the INPUT chain, specifying the protocol (-p) and the destination port (--dport).

Allow Web Traffic (HTTP/HTTPS)

Bash

Allow HTTP (Port 80)

```
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

Allow HTTPS (Port 443)

```
iptables -A INPUT -p tcp --dport 443 -j ACCEPT
```

Allow SSH (Port 22)

Bash

```
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

Allow Established Connections

If your default policy is to drop incoming traffic, you must allow packets that are part of an already established session (like a response from a website you visited):

Bash

```
iptables -A INPUT -m conntrack --ctstate  
ESTABLISHED,RELATED -j ACCEPT
```

3. Denying Specific Ports

If your default policy is set to ACCEPT, or if you want to explicitly block a specific service or port, you use the DROP target.

Block a Specific Port (e.g., FTP Port 21)

Bash

```
iptables -A INPUT -p tcp --dport 21 -j DROP
```

Block a Port from a Specific IP Address

If you want to block a specific malicious IP address from hitting a specific port (like port 22 for SSH):

Bash

```
iptables -A INPUT -p tcp -s 203.0.113.50 --dport 22 -j DROP
```

4. Managing and Saving Rules

View Current Rules

To see your active rules with line numbers (useful for deleting specific rules later):

Bash

```
iptables -L -v -n --line-numbers
```

Delete a Rule

To delete a specific rule, use the -D flag followed by the chain name and the line number from the command above:

Bash

Deletes the rule at line 3 in the INPUT chain

```
iptables -D INPUT 3
```

Making Rules Persistent

By default, iptables rules are cleared when the system reboots. To save them permanently:

- **Ubuntu / Debian:**

Bash

```
sudo apt install iptables-persistent
```

```
sudo netfilter-persistent save
```

- **CentOS / RHEL / Fedora:**

Bash

```
sudo service iptables save
```

❖ **Demonstrate blocking a port scan attempt**

A port scan (like an nmap TCP Connect or SYN scan) works by rapidly sending probes to hundreds of ports to see which ones respond. We can block this using two different approaches: **Static Blocking** (pre-emptively closing unneeded ports) and **Dynamic Rate Limiting** (detecting a rapid burst of connections and locking the scanner out).

The Setup

- **Attacker IP: 192.168.1.100**
- **Target Server IP: 192.168.1.50**

Approach 1: The "Drop and Silence" Method (Static Defense)

The simplest way to render a port scan useless is to change your default iptables behavior from REJECT to DROP.

When a port is closed and a scan hits it:

- REJECT sends back an explicit RST (Reset) or ICMP destination unreachable packet. The scanner immediately knows the host is alive, but the port is closed.
- DROP completely ignores the packet. The scanner gets nothing back and has to wait for a timeout.

By dropping traffic on all unused ports, an nmap scan slows down to a crawl and reveals no useful information.

Bash

1. Allow legitimate traffic first (e.g., SSH on port 22)

```
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

2. Set the default policy to DROP everything else

```
iptables -P INPUT DROP
```

The Result:

If the attacker runs a standard scan:

Bash

```
nmap 192.168.1.50
```

Instead of getting a quick list of closed ports, the scan will take a very long time, and nmap will eventually report that the host seems down or all ports are filtered.

Approach 2: Dynamic Port Scan Blocking (The "Recent" Module)

If you want to actively detect someone scanning multiple ports in a short timeframe and completely block their IP address for a set duration, you can use the iptables recent module.

Run these rules on the target server:

Bash

1. Create a rule named "PORTSCAN" that drops traffic from anyone on the list

```
iptables -A INPUT -m recent --name PORTSCAN --update --seconds 60 -j DROP
```

2. For any NEW connection attempts on unopened ports, log them and add the IP to the PORTSCAN list

```
iptables -A INPUT -p tcp -m state --state NEW -m recent --name PORTSCAN --set -j DROP
```

How This Works:

1. When a packet hits a closed port, the second rule triggers. It takes the attacker's IP address and adds it to a temporary tracking list called PORTSCAN. It then drops that specific packet.
2. If that same IP sends *any* more packets within 60 seconds, the first rule triggers. It catches them at the very top of the chain, updates their timer back to 60 seconds, and drops the traffic instantly.
3. The attacker is now effectively completely blocked from the entire server for a rolling 60-second window.

Approach 3: Blocking Christmas (Xmas) and Null Scans

Attackers often use modified TCP flags (like Null or Xmas scans) to bypass basic firewall rules. These packets have illegal combinations of flags that a normal application would never send.

You can explicitly block these common port-scanning techniques:

Bash

Block Null Scans (Packets with no flags set)

```
iptables -A INPUT -p tcp --tcp-flags ALL NONE -j DROP
```

Block Xmas Scans (Packets with FIN, PSH, and URG flags all turned on)

```
iptables -A INPUT -p tcp --tcp-flags ALL FIN,PSH,URG -j DROP
```

Block SYN-FIN Scans (An impossible combination used to probe firewalls)

```
iptables -A INPUT -p tcp --tcp-flags SYN,FIN SYN,FIN -j DROP
```

Verification: What the Attacker Sees

Before applying the rules, an nmap scan easily maps the target:

Plaintext

PORT STATE SERVICE

22/tcp open ssh

80/tcp closed http

After applying the **Dynamic/Static** rules, the attacker's scan output changes completely:

Plaintext

Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn

If they try to force it with -Pn (skip host discovery), they get:

Plaintext

All 1000 scanned ports on 192.168.1.50 are filtered

On the server, you can verify that the rules are actively catching the scan by checking the packet counters:

Bash

iptables -L INPUT -v -n